시계열 추론 실습 결과 보고서 (team2-2, 안내문 구성)

Oct 21, 2026 · @박민수

이 탭은 같은 내용을 안내문 순서(해결할 것 · 확인 기준 · 산출물 · 생각해 볼 점 · 심화)대로 다시 정리한 판이다. "해결한 것"의 번호는 안내문 "해결할 것"의 번호와 같다.

요약

과제 1~8의 확인 기준을 모두 넘었다. 과제 1·8은 실데이터, 과제 2~7은 고장 정답이 있는 합성 데이터로 판정했다. 실데이터 60분 CO_2 예측은 시간표가 없어 현재값 유지보다 18% 좋아지는 데 그쳤다(기준 30%). 서비스는 팀 Jetson에서 계속 돌고 있다(조회 API `http://172.27.91.139:8090`, 교육장 Wi-Fi).

항목	값
보드	Jetson Orin Nano 8GB, JetPack 7.2.1, 25W 모드, 클럭 고정 안 함 (팀원 작업과 같은 보드)
합성 데이터	직접 만든 생성기 seed 42, 2026-08-31 ~ 10-11 (6주). 학습 4주 · 검증 1주 · 시험 10-05 ~ 10-11 (고장 17건). 생성기 v2(냉방 정지가 온도를 3~5℃ 올리도록 고침)
실데이터	공용 InfluxDB `iot-bucket`에서 복사한 2025-08 ~ 2026-10-20 + 10-20부터 직접 수집. 시험 10-06 ~ 10-20. 시간표·고장 정답 없음
평가 지점	과제 2에서 정한 같은 지점(`eval_points.csv`)에서 모든 방법을 비교. 시험 기간은 학습·모델 선택에 쓰지 않음
코드	보드 `/home/team2-2/work/ts` (수집기는 `/home/team2-2/ingest`)

과제 1. 데이터 수집 체계

해결한 것

1. MQTT(`data.withio.net`)로 들어오는 센서 메시지를 받아 공간·항목·시각별로 저장하는 수집기를 2026-10-20 15:16부터 계속 돌린다. 저장은 두 곳에 한다. 보드 안 InfluxDB는 원본을 받아 두는 곳이고, PostgreSQL(Crowfoot에서 발급한 내 스키마)은 최종 저장소다.

2. 토픽, 시각이 든 위치, 공간·센서 이름, 값이 든 위치를 모두 설정 파일(`config.yaml`)에서 정한다. 코드를 고치지 않고 설정 파일만 바꿔서 대응한 일이 실제로 두 번 있었다. 냉난방 토픽을 추가한 것과 시각을 읽는 위치를 바꾼 것이다(자세한 내용은 과제 1 산출물의 "메시지 형식이 바뀐 상황과 대응").
3. 네트워크나 브로커가 끊기면 스스로 다시 붙는다. 보드를 재부팅하면 수집기가 자동으로 시작된다. 고정된 접속 이름을 쓰기 때문에 끊긴 동안의 메시지는 브로커가 보관했다가 다시 보내 준다. PostgreSQL에 못 쓴 값은 매시간 InfluxDB에서 다시 채운다.
4. 매일 09:05에 수집 현황 보고서를 자동으로 만든다. 공간·항목별 건수, 마지막 수신 시각, 커버율(정해진 시간 동안, 데이터가 있어야 할 칸 중에 실제로 데이터가 들어온 칸의 비율), 빈 구간과 그 원인이 들어 있다.

확인 기준

기준	결과	판정
브로커를 껌다 켜도 1분 안에 수집 재개	브로커는 외부 서버라 직접 끌 수 없어, 수집기 네트워크를 60초 끊는 것으로 대신했다. 복구 6초 뒤 재접속, 유실 0	통과
보드를 재부팅해도 자동 재개	약 44초 비었다가 재개. 브로커가 보관한 메시지를 받아 빈 동안의 값도 저장됐다	통과
24시간 커버율 95% 이상, 빠진 구간 원인 설명	1분마다 보내는 센서 9대 95.2~98.5%. 2분·10분마다 보내는 센서 5대는 자기 주기 기준 96.5~100%. 가장 긴 빈 구간 11분은 장애 시험(수집기 10분 정지)	통과

측정 조건: 실데이터, 커버율은 2026-10-20 15:17 ~ 10-21 09:05 자동 보고서.

산출물

- **수집 현황 보고**: 보드 `~/ingest/reports/coverage_YYYYMMDD.md`(매일 자동). 빈 구간의 원인은 센서가 보내는 일련번호(fCnt)로 나눈다. 번호가 건너뛰었으면 전송 중 유실, 번호가 이어지면 센서가 보내지 않은 것, 수집기 상태 기록이 비어 있으면 수집기 쪽 문제다.
- **1분 커버율의 한계**: 1분보다 길게 보내는 센서는 "1분 커버율"로는 95%를 넘을 수 없다(9.6~48.7%). 그래서 센서가 원래 보내는 간격(2분, 10분 등)으로 칸을 나눈 커버율도 함께 낸다. 이렇게 보면 이 센서들도 96.5~100%로, 보내야 할 때는 제대로 보냈다.
- **메시지 형식이 바뀐 상황과 대응**
  - 사례 1: 냉난방(xflow) 토픽을 추가했다. 메시지 모양(`{"ts": 밀리초, "values": {...}}`)이 센서와 달랐지만 `config.yaml`에 읽는 방법만 적어 처리했다.
  - 사례 2: 게이트웨이 한 대의 시계가 295초 빨랐다. 시각을 읽는 위치를 게이트웨이 시각(`time`)에서 네트워크 서버 시각(`rxInfo.0.nsTime`)으로 설정 한 줄만 바꿔 고쳤다.

생각해 볼 점

- **센서 시각과 수신 시각 중 무엇을 저장하나**: 이 센서들은 측정 시각을 보내지 않는다. 그래서 믿을 수 있는 가장 이른 시계인 네트워크 서버 수신 시각을 저장한다. 수집기가 받은 시각과의 차이(`rx_delay_s`)도 함께 남긴다. 네트워크가 30초 밀렸다 한꺼번에 와도 측정 순서와 간격이 그대로 유지된다. 수신 시각을 저장하면 밀린 값들이 한 시각에 몰려 기록된다.
- **수집과 학습이 같은 저장소를 쓰면**: 학습이 큰 범위를 읽는 동안 쓰기가 늦어지고, 학습 중에 데이터가 바뀌어 같은 결과를 다시 낼 수 없다. 그래서 수집기는 쓰기만 하고, 학습은 내보낸 파일이나 PostgreSQL을 읽기만 한다.

심화(B)

- **같은 메시지가 두 번 오는 경우**: 센서는 메시지 고유번호(`deduplicationId`), 냉난방은 (토픽, 시각)으로 거르고, PostgreSQL 기본 키가 한 번 더 막는다. 같은 냉난방 메시지가 두 번씩 들어오는 문제를 실제로 찾아 고쳤다. 넓은 토픽(xflow/#)과 그 안에 포함된 좁은 토픽을 따로 구독해서, 브로커가 같은 메시지를 구독마다 한 번씩 보냈기 때문이다. 구독을 넓은 토픽 하나로 줄였다.
- **1년 뒤 저장소 크기와 보관 정책**: 지금처럼 원본을 계속 쌓으면 1년에 약 9.8GB, 3년에 약 29GB가 된다. 원본은 30일(진단용 정보는 7일), 5분 집계는 400일, 1시간 집계는 영구 보관하는 정책을 설계했다. 이 정책이면 1년 뒤 약 1GB에서 거의 멈추고, 영구 보관하는 1시간 집계만 해마다 약 34MB씩 늘어난다. 학습에 쓰는 5분 표는 5분 집계만으로 만들 수 있다. 설계만 했고 실제로 지운 데이터는 없다.

저장소 실측값 (최근 24시간)

항목	값
InfluxDB 하루 값 개수	235,896개 (센서 메시지, 환경값, 냉난방, 추론 결과, 수집기 상태)
InfluxDB 값 하나의 크기 (압축 후)	5.83바이트
PostgreSQL 하루 행 수	43,062행
PostgreSQL 행 하나의 크기 (색인 포함)	589바이트

1년 크기 추정

저장소	정책 없음, 1년	정책 없음, 3년	정책 적용, 1년	정책 적용, 3년
InfluxDB	0.50GB	1.5GB	0.10GB	0.11GB
PostgreSQL	9.26GB	27.8GB	0.89GB	0.98GB
합계	**약 9.8GB**	**약 29GB**	**약 0.98GB**	**약 1.09GB**

측정 조건: 실데이터, 2026-10-21 16시경 측정, 수집 시작 뒤 약 1일 분량. InfluxDB 압축 후 크기는 과거 기록 버킷(445일, 677만 값)에서 잰 값이다. 집계 테이블의 행 크기(120바이트)는 아직 만들지 않아 가정한 값이다.

PostgreSQL이 대부분을 차지하는 이유는 센서 메시지 1건이 환경값 행과 메시지 원본 행 두 줄로 저장되고, 압축도 되지 않기 때문이다. 그래서 정책은 PostgreSQL 원본을 짧게 두는 데 초점을 두었다.

과제 2. 데이터 이해와 기준선

해결한 것

1. **정제 규칙**을 여섯 가지로 정했다. 학습과 서비스가 같은 코드(`prep.py`)를 쓴다.
   a. 실제로 나올 수 없는 값(CO_2 300ppm 미만·5,000ppm 초과, 온도 -10℃ 미만·50℃ 초과, 습도 0~100% 밖)은 빈칸으로 바꾼다. 바깥 공기의 CO_2가 약 420ppm이라 실내가 300ppm보다 낮을 수 없고, 0ppm·65535ppm은 센서 오류 값이다. 이런 값이 하나만 섞여도 그 5분 평균이 수백 ppm 틀어진다. 지우지 않고 빈칸으로 두면 3번 규칙이 처리한다. (합성 6주에서 CO_2 75건, 온도 83건)
   b. 센서에서 들어오는 값을 5분씩 묶어 평균을 낸다. 예를 들어 10:01~10:05의 평균을 "10:05 값"으로 부른다. 묶음에 끝나는 시각의 이름을
   c. 붙이기 때문에, "10:05 값"에는 10:05까지 이미 들어온 값만 들어 있다. 센서마다 값을 보내는 주기가 다르다(강의실 1분, 회의실 2분, 라운지 5분). 5분으로 묶으면 가장 느린 라운지도 칸마다 값이 하나씩 들어가고, 60분 예측도 5분 간격이라 단위가 맞는다. 1분마다 보내는 센서는 5분 칸에 값이 5개 들어가, 평균을 내면 측정 흔들림이 절반 아래로 줄어든다(합성 데이터 기준 8ppm → 3.6ppm).
   d. 값이 빠진 구간이 15분 이하면 앞뒤 값을 이어서 메우고, 메운 값이라는 표시를 붙인다. 15분보다 길게 빠지면 메우지 않고 빈칸으로 둔다. 실제 값을 일부러 지우고 메워 보니, 15분 빈칸은 오차가 8~10ppm(평일 낮 14~19ppm)이었지만 30분이면 평일 낮 22~37ppm, 60분이면 40~69ppm으로 커졌다. 10분과 15분은 메워지는 빈칸 비율이 거의 같고(93~99% vs 95~100%) 오차 차이도 작다. 30분부터는 오차가 두 배 가까이 커지므로, 그 직전인 15분을 기준으로 했다.
   e. 센서가 여러 대인 공간(실습실 8대, 사무실 2대)은 센서마다 값을 따로 정리한 뒤 평균을 낸다. 그냥 평균을 내면 5분 칸마다 들어온 센서가 바뀌어 값이 괜히 오르내린다(아래 예시 표). 센서별로 정리하자 실데이터에서 "5분 뒤에도 그대로" 예측의 온도 오차가 0.07℃에서 0.02℃로 줄었다.
   f. 값의 크기를 맞추는 정규화(평균을 빼고 표준편차로 나누기)는 학습 기간 데이터로만 계산한다. 시험 기간까지 넣으면 미래 정보가 학습에 섞이기 때문이다.
   g. 튀 값이나 멈춰서 같은 값만 보내는 고장은 여기서 지우지 않는다. 정상 범위 안의 값이라 앞뒤 흐름을 봐야 고장인지 알 수 있고, 이걸 찾는 것이 과제 4의 일이다. 여기서 지우면 탐지기가 고장을 잡는지 시험할 수 없다.
2. **분석 간격은 5분**으로 정했다. 1분을 그대로 쓰면 2분·5분마다 보내는 센서는 칸의 50~80%가 비어, 대부분을 빈칸 처리해야 한다. 반대로 10분 이상으로 묶으면 수업 시작 뒤 CO_2가 오르는 모습(보통 1.5~8분 안에 반응)이 뭉개진다. 5분은 모든 센서가 칸마다 값을 갖고 반응도 1~2칸 안에 잡히는 가장

짧은 간격이다.

3. **학습·검증·시험은 시간 순서대로** 나눴다(학습 4주 → 검증 1주 → 시험 1주). 무작위로 섞으면 안 되는 이유는 세 가지다. 이웃한 5분 칸끼리 값이 거의 같아서, 섞으면 시험 칸 바로 옆 칸이 학습에 들어가 모델이 "옆 칸을 외워" 점수가 부풀려진다. 60분 뒤를 맞히는 문제라 학습 샘플의 정답이 시험 샘플의 입력과 겹친다. 실제 운영에서도 항상 과거로 학습하고 미래를 예측한다.
4. **데이터를 진단**했다(아래 표).
   - 빠진 값은 대부분 게이트웨이가 2~12분 끊긴 것이다. 남는 빠진 값은 공간마다 1~3번 있는 몇 시간짜리 끊김뿐이다. 라운지는 5분마다 보내서 메시지 하나만 빠져도 칸이 통째로 빈다.
   - 평일 시간대별 패턴에 수업·점심·퇴실이 그대로 보인다. 강의실은 9시에 420→1,000ppm까지 오르고, 점심에는 강의실이 떨어지고 라운지가 1,370ppm까지 오른다. 18시 이후 강의실은 1시간 안에 500ppm으로 떨어진다.
   - 일정이 바뀐 뒤 CO_2는 강의실에서 오를 때 1.5~3분, 내려갈 때 5~6분 만에 반응한다. 회의실은 8분으로 가장 느리고 들십날십하다(예약보다 늦게 오고 인원이 2~8명).
   - 일정과 실제가 어긋난 날: 휴강 5일을 CO_2로 찾았고 모두 맞았다. 회의실 노쇼는 19건으로 추정해 진짜 10건을 모두 잡았지만, 9건은 직전 회의가 끝나 CO_2가 이미 높아서 오르지 않은 경우를 노쇼로 잘못 봤다.
5. **기준선 3가지**(현재값 유지, 최근 1시간 평균, 어제 같은 시각)의 5·15·30·60분 뒤 오차를 재다(아래 표). 가장 강한 것은 현재값 유지다.

정제 규칙 4번 예시 (센서 A는 늘 23.0℃, B는 늘 23.4℃로 재는 경우)

5분 칸	들어온 센서	그냥 평균 (℃)	센서별 정리 (℃)
10:05	A만	23.0	A 23.0 + B 23.4(직전 값) → 23.2
10:10	B만	23.4	A 23.0(직전 값) + B 23.4 → 23.2
10:15	A, B	23.2	23.2

4번 빠진 값·이상값 진단 (합성 데이터 6주치)

공간	보내는 주기	메시지 수신률	5분 칸 빈칸 비율 (메우기 전 → 후)	지운 이상값 (CO_2 / 온도)	가장 길게 빠진 구간
강의실 A	1분	98.9%	0.8% → 0.6%	33 / 26	350분
강의실 B	1분	99.1%	0.6% → 0.4%	26 / 33	225분
회의실	2분	99.2%	0.6% → 0.4%	12 / 17	215분
라운지	5분	98.3%	1.8% → 1.2%	4 / 7	320분

5번 기준선 오차 (MAE, 시험 기간)

데이터 · 대상	몇 분 뒤	현재값 유지	최근 1시간 평균	어제 같은 시각
합성 CO_2 (ppm)	5 / 15 / 30 / 60	**10.4 / 19.7 / 30.1 / 44.0**	26.9 / 32.6 / 40.1 / 51.6	94.7 / 95.2 / 95.9 / 97.3
합성 온도 (℃)	5 / 15 / 30 / 60	**0.09 / 0.24 / 0.46 / 0.87**	0.48 / 0.62 / 0.83 / 1.23	1.31 / 1.31 / 1.31 / 1.30
실데이터 CO_2 (ppm)	5 / 15 / 30 / 60	**7.6 / 14.5 / 27.2 / 46.4**	23.8 / 30.0 / 42.9 / 60.8	122.4 / 122.3 / 135.0 / 135.1
실데이터 온도 (℃)	60	**0.13**	0.18	0.50

측정 조건: 합성 시험 2026-10-05 ~ 10-11, 실데이터 시험 10-06 ~ 10-20, 같은 평가 지점(합성 60분 6,046개). 평가 지점은 지금 값과 정답이 모두 실제 측정값이고, 4시간 전부터 정답 시각까지 고장이 없는 칸이다.

확인 기준

기준	결과	판정
정규화 값을 학습 기간으로만 계산	학습 기간의 실제 측정값(메운 값 제외)으로만 계산. CO_2 524 ± 213ppm, 온도 24.3 ± 2.8℃	통과
기준선과 이후 모델을 같은 시점·조건에서 평가	평가 지점을 `eval_points.csv`로 고정해 과제 3~6이 그대로 쓴다	통과
가장 강한 기준선과 이유 설명	현재값 유지. 아래 “생각해 볼 점”	통과

산출물

- **데이터 진단서**: 위 “해결한 것” 1~4와 표. 그래프 3개(평일 시간대별 패턴, 하루 예시, 기준선)는 `report/diagnosis.md`에 있다.
- **기준선 성능표**: 위 5번 표.

생각해 볼 점

- **현재값 유지가 가장 강한 이유**: CO_2와 온도는 몇 분 사이에 크게 바뀌지 않는다. 최근 1시간 평균은 같은 정보를 쓰면서 과거로 끌려가서, 수업 중처럼 계속 오르는 구간에서 항상 늦는다.

- **어제 같은 시각이 가장 약한 이유**: 어제와 오늘이 다른 날이 많다. 월요일의 어제는 일요일이고, 휴강이 끼면 통째로 틀린다. 반대로 **시간표가 매일 같은 강의실에서 평일이 이어질 때는 강하다**(시험 주 목요일 낮 60.7ppm vs 현재값 유지 97.3ppm). 그래서 과제 3에서는 어제 값 대신 일정표를 직접 입력으로 넣었다.
- **고장이 섞인 구간**: 합성 데이터의 고장은 시험 기간(마지막 1주)에만 있어 학습 기간에는 없다. 예측 평가에서는 고장 구간을 뺐다. 고장이 섞이면 예측 실력이 아니라 고장 때문에 오차가 커지기 때문이다. 고장은 과제 4에서 따로 평가한다.

심화(B)

안내문에 과제 2의 심화 항목은 없다.

과제 3. 60분 예측

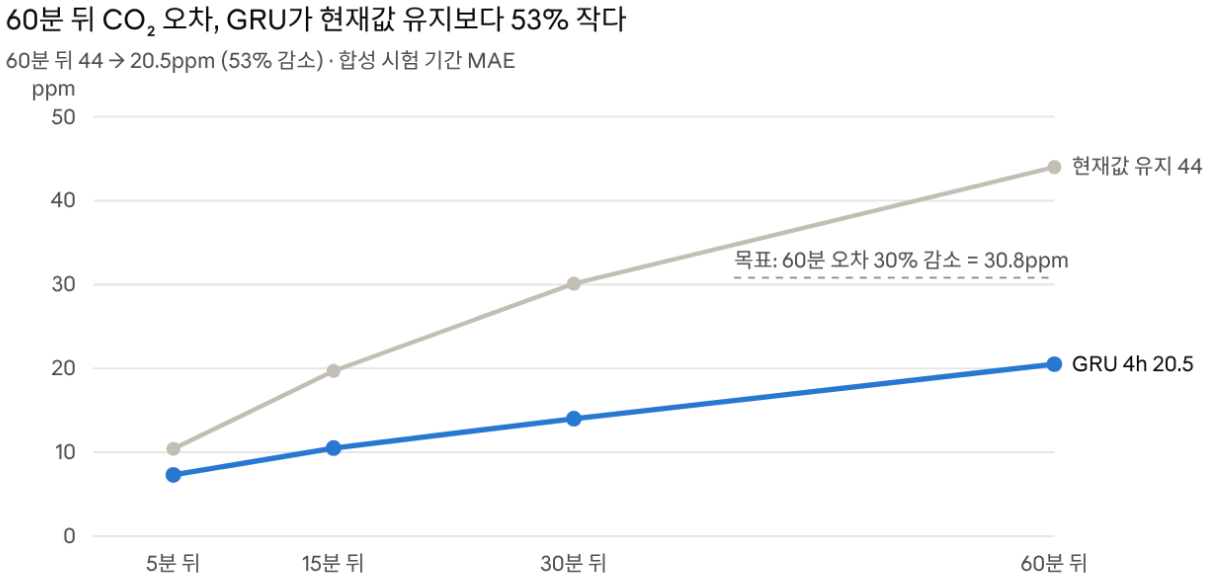
해결한 것

1. **입력과 출력을 설계했다.**
   - 과거 길이: 1·2·4시간을 비교했고, 검증 성적으로 4시간을 골랐다.
   - 넣는 정보: CO_2·온도·습도, 메운 값 표시, 시각·요일, 어느 공간인지, 일정(예정 인원). 앞으로 60분 동안의 예정 인원과 시각도 미리 알 수 있어 따로 넣는다.
   - 출력: 60분을 한 번에(5분 간격 12칸 × CO_2·온도) 낸다. 한 칸씩 반복하면 앞 칸의 오차가 뒤 칸에 쌓이기 때문이다. 모델은 지금 값에서 얼마나 바뀔지를 맞힌다.
2. **딥러닝 모델 3종(GRU, LSTM, TCN)**을 과거 길이 3가지로 학습했다. 모델은 검증 기간의 60분 CO_2 오차로만 골랐다.
3. **기준선과 같은 지점에서 비교**했다(아래 표와 그래프). 고른 GRU 4시간은 60분 뒤 CO_2 오차가 20.5ppm으로 현재값 유지(44.0)보다 53% 작다.
4. **공간별로 나눠 봤다.** 회의실은 평균 오차가 가장 작지만(17.5ppm) 대부분 비어 있어서다. 사용 중일 때만 보면 92.2ppm으로 가장 어렵다. 예약 시각과 실제 시작이 어긋나고(중앙값 8분 늦음), 노쇼가 15%이고, 인원이 2~8명으로 들쑥날쑥하기 때문이다.
5. **일정 정보의 가치를 재다.** 같은 GRU 4시간에서 일정을 빼면 60분 CO_2 오차가 20.5에서 23.7ppm으로 나빠진다. **일정 정보를 넣으면 60분 예측 오차가 3.2ppm(14%) 줄어든다.** 사용 중인 회의실만 보면 177.5에서 92.2ppm으로 절반이 된다.

3번 모델 비교 (합성 시험, MAE)

모델	과거	CO_2 5분	15분	30분	60분	온도 60분	검증 CO_2 60분
현재값 유지	—	10.4	19.7	30.1	**44.0**	**0.87**	—
GRU	1시간	7.6	11.0	14.2	19.8	0.29	22.0
GRU	2시간	7.5	10.9	15.2	20.1	0.30	21.7
GRU (선택)	**4시간**	7.3	10.5	14.0	**20.5**	**0.31**	**21.1**

모델	과거	CO_2 5분	15분	30분	60분	온도 60분	검증 CO_2 60분
LSTM	1·2·4시간	—	—	—	21.1 · 22.9 · 22.1	0.28~0.29	23.4~24.5
TCN	1·2·4시간	—	10.4~11.3	13.3~14.8	19.0 · 18.4 · 20.1	0.47~0.69	21.6~22.2



4번 공간별 60분 CO_2 (GRU 4시간)

공간	현재값 유지	GRU	GRU, 사용 중 시간만
강의실 A	46.5	23.3	64.3
강의실 B	64.9	23.3	66.4
라운지	46.4	18.4	56.4
회의실	21.3	17.5	**92.2**

측정 조건: 합성 시험 2026-10-05 ~ 10-11, 과제 2와 같은 평가 지점, 학습은 GPU. 실데이터 결과는 맨 아래 "실데이터 재평가".

확인 기준

기준	결과	판정
60분 뒤 CO_2 오차가 현재값 유지보다 30% 이상 작다	44.0 → 20.5ppm, 53% 작다	통과
온도 60분 예측이 현재값 유지보다 좋다	0.87 → 0.31°C	통과

기준	결과	판정
시험 기간을 학습·모델 선택에 쓰지 않았다	모델·과거 길이는 검증 기간 오차로만 골랐다	통과

산출물

- **모델 비교표**: 위 3번 표와 그래프.
- **공간별 오차 분석과 원인**: 위 4번.
- **일정 포함/제외 비교와 결론**: 위 5번. “일정 정보를 넣으면 60분 예측 오차가 3.2ppm 줄어든다.”

생각해 볼 점

- **회의실이 어려운 이유**: 회의가 언제 시작할지, 몇 명이 올지는 과거 센서값에 전혀 나타나지 않는다. 이 정보는 회의실 예약에서 얻는다. 예약을 넣자 사용 중 오차가 절반이 됐다.
- **지금 값 vs 변화량 예측**: 변화량을 맞히게 했다. 지금 값을 통째로 맞히게 하면 모델이 “지금 값을 그대로 내는 법”부터 배워야 하고, 짧은 시점에서 현재값 유지도 못 이기기 쉽다. 변화량을 맞히면 바뀌는 부분만 배우면 된다. 두 방식을 숫자로 비교하지는 않았다.
- **오차가 가장 큰 순간**: 평일 09시(수업 시작) 95ppm, 12시(점심) 80ppm이다. 밤에는 6~10ppm이다. 시작 시각은 일정으로 알 수 있지만 몇 명이 올지(결석·휴강)는 알 수 없어서 이 순간의 오차는 남는다.

심화(B)

- **과거 길이 × 모델 조합**: 검증 기준 1등은 GRU 4시간이다. TCN 2시간은 시험 CO_2가 가장 좋지만(18.4) 온도가 두 배 나빠서(0.60) 고르지 않았다. 과거 길이 1·2·4시간 사이 차이는 1ppm 안쪽이다. 60분 앞을 맞히는 데는 최근 1시간이면 충분하고, 그 이상은 시각·일정 입력이 대신한다.
- **미래에 알 수 있는 정보 넣기**: 앞으로 60분의 예정 인원을 입력에 넣었다. 사용 중인 회의실 오차가 177.5에서 92.2ppm으로 줄었다.
- **처음 보는 공간**: 회의실을 학습에서 빼도 회의실 오차는 16.6ppm으로 거의 같다. 강의실 B를 빼면 41.2ppm(포함 시 23.3)으로 나빠진다. 비슷한 강의실이 하나뿐이면 인원 규모(18명 vs 26명) 차이를 배울 곳이 없다.

과제 4. 이상 탐지

해결한 것

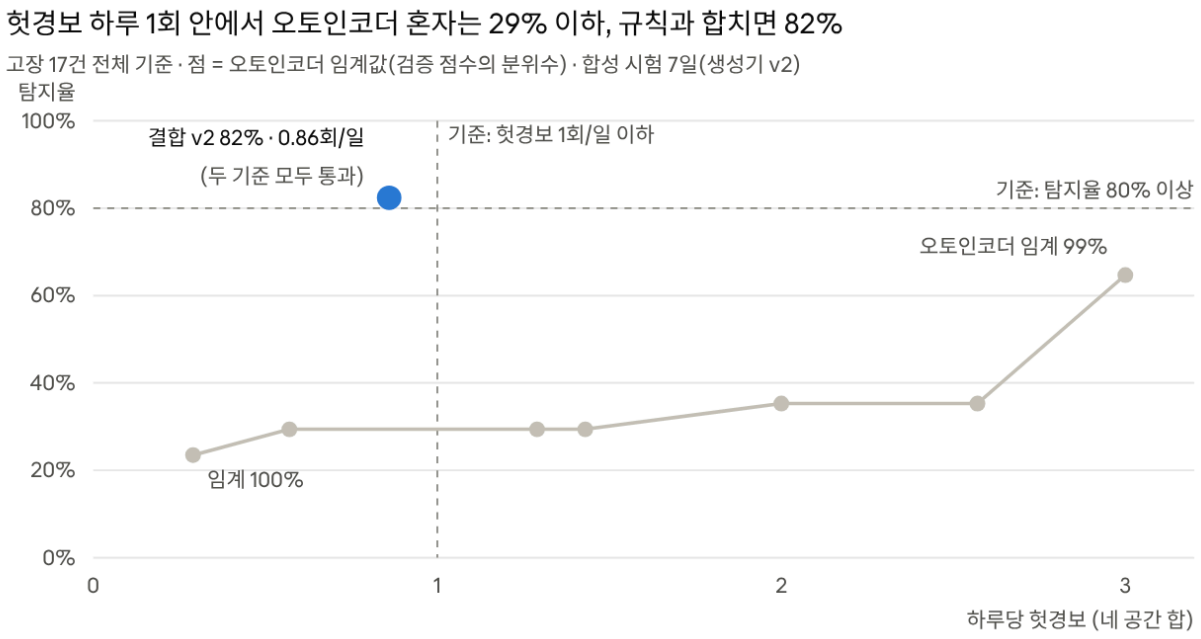
1. **정상만 보고 이상을 찾는 모델**로 오토인코더를 학습했다. 최근 1시간을 넣고 다시 그려 보게 하여, 잘 못 그리는 구간을 이상으로 본다. 학습에는 고장이 없는 데이터만 썼다.

2. **경보 기준(임계값)은 검증 기간(고장 없음)의 헛경보 수**로 정했다. 기준을 높일수록 헛경보가 줄고 탐지율도 줄어든다. 오토인코더 기준을 상위 1%에서 최댓값으로 올리면 헛경보는 하루 3.0회에서 0.29회로, 탐지율은 65%에서 24%로 떨어진다(아래 그래프).
3. **규칙 기반 탐지**를 만들어 모델과 비교했다.
   - 급변: 값 하나가 직전 세 값과 다음 값 모두에서 크게 벗어남
   - 값 고정: 세 값이 30분 동안 똑같음
   - 영점 틀어짐: 일정 없는 시간에 CO_2가 평평한데 바깥 공기보다 100ppm 넘게 높음
   - 정상 범위: 직전 7일의 같은 공간·같은 시각 최댓값을 넘음
4. **고장 종류별로 평가**했다. 탐지율, 탐지 지연, 하루 헛경보 수, 종류별 탐지 결과를 아래 표에 냈다.
5. **경보가 어느 센서 때문인지** 가리킨다. 결합 탐지기가 가리킨 원인 센서는 잡은 고장에서 모두 맞았다(100%). 오토인코더 혼자서는 60%였다.

4번 탐지 성적표 (합성 시험 7일, 고장 17건)

탐지기	탐지율 (전체 17)	탐지율 (관측 가능 15)	평균 지연	헛경보/일	튀 값	값 고정	영점	냉방 정지	환기 정지
오토인코더	29%	33%	218분	1.43	2/6	1/3	2/2	0/3	0/3
규칙(급변·고정·영점)	53%	60%	84분	0.14	5/6	2/3	2/2	0/3	0/3
정상 범위(최근 7일)	41%	47%	119분	0.14	0/6	0/3	2/2	2/3	3/3
예측 오차(심화)	24%	27%	181분	0.43	0/6	0/3	2/2	1/3	1/3
결합 v2 (오토인코더 높은 기준 + 규칙 + 정상 범위)	**82%**	93%	64분	**0.86**	5/6	2/3	2/2	2/3	3/3
결합 v3 (v2 + 예측 오차)	88%	100%	62분	1.14	5/6	2/3	2/2	3/3	3/3

측정 조건: 합성 시험 2026-10-05 ~ 10-11, 생성기 v2. 임계값은 모두 검증 기간 헛경보로 정했다. v1 생성기는 냉방 정지가 온도를 올리지 않는 버그가 있었고, 그때 결합 v2 탐지율은 71%였다.



확인 기준

기준	결과	판정
모델과 규칙을 합쳐 이벤트 탐지율 80% 이상	결합 v2, 고장 17건 중 14건(82%)	통과
헛경보 하루 1회 이하 (전체 공간 합)	하루 0.86회	통과
모델이 못 잡는 고장과 규칙이 못 잡는 고장을 찾아 이유 설명	모델은 값 고정, 규칙은 냉방·환기 정지(아래 생각해 볼 점)	통과

산출물

- **탐지 성적표** (모델 / 규칙 / 결합 × 고장 종류): 위 4번 표.
- **임계값에 따른 탐지율·헛경보 변화**: 위 그래프. 경보 기준은 “탐지율”보다 “운영자가 하루에 감당할 알림 수”로 먼저 정하고, 그 안에서 탐지율을 올리는 순서가 맞다.
- **운영 제안**
  - 강의실: 정상 범위 규칙(CO_2 +200ppm, 온도 +1.5℃)으로 시설 담당에게 알린다.
  - 모든 공간: 값 고정·급변·데이터 끊김은 IT(센서 관리) 담당에게 알린다.
  - 영점 틀어짐: 실시간 경보 대신 “새벽 CO_2가 바깥 공기보다 높음” 일일 점검 보고로 보낸다. 잡는 데 2~5시간이 걸려 실시간 알림의 의미가 적다.

생각해 볼 점

- **정상 범위의 값에서 멈추면 왜 모델이 못 잡나**: 멈춘 값은 흔들림이 없어 오토인코더가 “조용한 정상”처럼 오히려 더 잘 그려낸다. 값 고정 3건 중 2건을 놓쳤다. 규칙은 “세 값이 똑같다”로 바로 잡는다(지연 29분).
- **규칙이 못 잡는 것**: 냉방·환기 정지 같은 설비 고장이다. 값은 정상적으로 움직이면서 “평소보다 높을”뿐이라, 평소와 비교하는 정상 범위 규칙과 예측 오차가 잡는다.
- **빈 방에서 환기가 멈추면**: 사람이 없으면 CO_2가 오르지 않아 센서로는 알 수 없다. 바로 알릴 필요도 적다. 다음 사용자가 들어오면 평소보다 빨리 오르므로 그때 잡힌다.
- **정답이 없는 실데이터에서 확인하는 법**: 일부러 만든 고장으로 확인했다. 수집기를 10분 멈추자 데이터 끊김 경보가 정확히 울렸다. 합성 데이터로 학습한 오토인코더는 사무실에서 밤새 476분 울렸는데, 실데이터 분포가 달라서다. 실데이터로 다시 학습해야 한다.
- **남은 사각지대**: 회의실 냉방 정지 1건. 회의실은 평소에도 오후에 28℃ 가까이 올라 +4℃가 평소 범위에 묻힌다. 냉난방 실내기 상태(xflow)를 함께 보면 고장 순간에 바로 알 수 있다. 이 값은 실데이터에서 이미 수집 중이다.
- **관측할 수 없었던 2건**: 라운지 값 고정은 고장 동안 게이트웨이가 끊겨 메시지가 0건이었다. 라운지 튀값은 5분마다 보내는 센서의 보고 사이 1분에 생겨 기록되지 않았다.

심화(B)

- **예측 오차를 이상 점수로 쓰는 탐지기**: 30분 전 예측보다 크게 높은 상태가 30분 이어지면 경보한다. 혼자서는 24%지만 냉방·환기 정지를 잡아, 결합 v3는 탐지율 88%까지 오른다. 다만 헛경보가 하루 1.14회로 기준을 넘는다. 오토인코더보다는 탐지율이 낮고(24% vs 29%) 헛경보는 적다(0.43 vs 1.43).
- **영점 틀어짐 전용 방법**: “사람이 없는 시간의 CO_2는 바깥 공기와 비슷해야 한다”를 이용했다. 일정 없는 시간에 CO_2가 평평한데 바깥 공기보다 100ppm 넘게 높으면 경보한다. 영점 틀어짐 2건을 모두 잡았다.

과제 5. 상태 추정

해결한 것

1. **정답을 정의했다.**
    - 사용 여부: 수업 시간표·회의실 예약으로 정답을 만들고, 공간마다 시험 기간 평일 이틀(9~18시)을 직접 관찰한 것으로 정답 품질을 먼저 확인했다. 직접 관찰은 합성 데이터의 실제 인원으로 대신했다. 일정 기반 정답은 22.5%가 틀렸다.
    - 환기 경보: “지금은 1,000ppm 미만이고 30분 안에 1,000ppm 이상이 되는 시점”을 양성으로 봤다. 이미 넘은 시점은 뺐다. 그때는 “미리” 알리는 것이 아니고, 넣으면 모델이 “지금 높으면 경보”라는 쉬운 문제만 배운다.

2. **사용 여부 추정에는 일정을 입력으로 쓰지 않았다.** 정답이 일정에서 나왔는데 입력에도 넣으면, 모델이 “일정에 있으면 사용”이라는 지름길을 외운다. 점수는 높게 나오지만 정작 찾고 싶은 노쇼와 휴강을 하나도 못 찾는다.
3. **분류 모델 2종을 학습했다.** 사용 여부는 GBDT(센서값만), 환기 경보는 MLP다. 환기 경보의 경보 기준(0.83)은 검증 기간에서 정했다.
4. **불균형을 다뤘다.** 환기 경보 양성은 전체의 1.6%라서, 아무 경보도 안 내도 정확도가 98.4%다. 그래서 정확도 대신 재현율(실제 양성 중 잡은 비율)과 macro-F1(두 분류의 F1 평균)로 평가했다.
5. **단순 규칙과 비교했다.** 사용 여부는 “최근 15분 CO_2가 30ppm 이상 올랐으면 사용중”, 환기 경보는 “지금 CO_2가 850ppm 이상이면 경보”다(아래 표).

사용 여부 (직접 관찰한 시간대 723칸)

방법	정확도	F1	빈 방을 사용중으로 본 비율
일정 그대로 (정답 품질 확인)	0.775	0.749	21.5%
규칙: 15분 CO_2 +30ppm	0.784	0.693	3.2%
분류 모델 (센서만)	**0.887**	**0.854**	**0.5%**

환기 선제 경보 (시험 기간 6,448칸, 양성 1.6%)

방법	정확도	재현율	정밀도	macro-F1
항상 ‘경보 없음’	0.984	0	—	0.496
규칙: 지금 CO_2 ≥ 850	0.987	0.686	0.565	0.806
분류 모델 (기준 0.83)	0.992	**0.941**	0.686	**0.895**
과제 3 예측 모델 (30분 안 최대 예측 ≥ 1,000)	0.994	0.839	0.796	0.907

혼동 행렬 (분류 모델, 행 = 실제, 열 = 모델 판단)

사용 여부	비어있음으로 판단	사용중으로 판단
실제 비어있음	402	2
실제 사용중	**80**	239

환기 경보	경보 안 함	경보
실제 음성	6,302	44
실제 양성	6	96

측정 조건: 합성 시험 2026-10-05 ~ 10-11. 환기 경보 기준은 검증 기간에서 정했다.

확인 기준

기준	결과	판정
사용 여부: 직접 관찰 기준 정확도·F1, 빈 방→사용중 오류율, 단순 규칙보다 좋음	정확도 0.887, F1 0.854(규칙 0.693), 빈 방→사용중 0.5%	통과
환기 경보: 재현율 85% 이상, 단순 규칙보다 macro-F1 높음	재현율 94.1%, macro-F1 0.895(규칙 0.806)	통과
혼동 행렬로 가장 많이 헷갈리는 경우 설명	아래 생각해 볼 점	통과

산출물

- **두 분류기의 성적표와 혼동 행렬**: 위 표.
- **일정 기반 정답과 직접 관찰의 차이**: 일정 기반 정답은 22.5%가 틀린다. 휴강이 끼어 있던 강의실 A는 “일정=사용, 실제=빈 방”이 81칸이고, 라운지는 일정 없이 쓴 것이 41칸이다.
- **가장 많이 틀리는 상황**: “사용중인데 빈 방”으로 본 경우다(80칸). 그중 16칸이 사람이 막 들어온 직후 15분이다. 라운지는 머무는 시간이 5~40분으로 짧고 인원이 적어 가장 어렵다.

생각해 볼 점

- **막 나간 직후와 막 들어온 직후**: 막 들어온 직후가 더 어렵다. 사람이 들어와도 CO_2가 오르기까지 1.5~8분이 걸리고, 5분 평균이 그 변화를 더 늦춘다. 막 나간 직후는 CO_2가 아직 높아도, 모델이 “떨어지는 속도”를 보고 구분한다. 그래서 빈 방을 사용중으로 본 경우는 2칸(0.5%)뿐이다.
- **큰 강의실을 적은 인원이 쓸 때**: 같은 인원이라도 방이 클수록 CO_2가 느리게 오른다. 그래서 방이 클수록 구별할 수 있는 최소 인원이 커진다. 이 한계를 숫자로 재지는 않았다.
- **환기 경보의 재현율 vs 정밀도**: 재현율을 우선했다. 놓치면 수업 중 머리 아픈 상태가 되고, 헛경보는 창문을 조금 일찍 여는 비용이다. 다만 정밀도가 0.69라 경보 3번 중 1번은 헛경보다. 이게 더 낮아지면 사람들이 알림을 무시하게 되므로, 경보는 30분에 한 번으로 묶어 보낸다(과제 8).

- **예측 모델로 대신할 수 있나**: 과제 3 예측 모델로 판단하면 macro-F1은 비슷하게 높지만(0.907) 재현율이 84%로 기준 아래다. 서비스에서는 둘 다 계산해, 분류기는 경보에, 예측은 "몇 분 뒤"를 알려 주는 데 쓴다.

심화(B)

- **처음 보는 공간**: 한 공간을 학습에서 빼고 시험했다. 강의실 A·B와 회의실은 F1 0.79~0.90으로 바로 쓸 만하다. 라운지는 0.58로 그 공간 데이터가 필요하다.
- **회의실 노쇼와 예약 없는 사용 찾기**: 사용 여부 분류 모델의 판단으로 두 규칙을 만들었다. 노쇼는 "예약 시작 뒤 15분 동안 계속 비어있음", 예약 없는 사용은 "15분 이상 사용중인데 어떤 예약과도 겹치지 않음"이다. 직접 관찰 대신 합성 데이터의 실제 인원과 비교했다(아래 표). 노쇼는 실제 3건을 모두 잡고 헛경보는 1건으로, CO_2 상승 규칙(헛경보 3건)보다 절반 아래로 줄었다. CO_2 규칙이 틀렸던 "직전 회의 바로 뒤" 경우를 모델은 맞혔다. 남은 헛경보 1건은 첫 사람이 12분 뒤에 들어온 회의다.

노쇼 찾기 (회의실, 검증+시험 2주)

규칙	예약	실제 노쇼	노쇼로 판정	맞힘	놓침	헛경보
모델 판단, 15분 비어있음	27	3	4	3	0	**1**
(기존) CO_2 30분 안 +50ppm 없음	27	3	6	3	0	3

예약 없는 사용 찾기 (검증+시험 2주)

공간	실제 건수	판정	맞힘	놓침	헛경보
회의실	1	1	1	0	0
라운지	5	0	0	**5**	0
강의실 A·B	0	0	0	0	0

측정 조건: 합성 데이터, 시험 주간에 노쇼가 2건뿐이라 모델 학습에 쓰지 않은 검증 주간을 합쳐 2주(09-28 ~ 10-11)로 봤다. 정답은 실제 인원(직접 관찰 대신).

라운지에서 놓친 5건은 모두 저녁이나 이른 아침에 2~3명이 20~40분 머문 경우다. 라운지(180m³)는 넓어서 2~3명으로는 CO_2가 거의 오르지 않는다. 헛경보를 더 줄이려면 노쇼 알림을 두 단계(15분 비어있음 → 확인 요청, 30분 → 예약 해제)로 두고, 넓은 공간에는 동작 감지(PIR) 센서를 함께 쓰는 것이 좋다. 다만 노쇼가 3건뿐이라 표본이 작다.

과제 6. 사전학습 모델 비교

해결한 것

1. **사전학습 모델 3종(Chronos-Bolt tiny·small·base)**으로 시험 기간 60분 CO_2를 추가 학습 없이 예측했다. 넣는 과거 길이(1시간~일주일)는 검증 기간 오차로 골랐고, 세 모델 모두 일주일이 뽑혔다.
2. **같은 지점에서 기준선·직접 학습한 모델과 비교**했다(아래 표).
3. **예측 구간(10~90%)이 실제로 얼마나 맞는지** 재다. 이상적이면 80%가 구간 안에 들어와야 한다. base 79%, small 76%, tiny 67%다.
4. **비용을 재다.** 보드 GPU에서 1회 추론 시간은 35~84ms, GPU 메모리는 가중치만 33~783MB다.
5. **채택 의견**: 주 예측 모델로는 채택하지 않는다. 과거를 일주일 주면 base도 현재값 유지보다 37% 좋지만(27.8ppm), 직접 학습한 GRU(20.5ppm)보다 7ppm 나쁘고 250배 느리다. small·과거 일주일만 예측 구간 표시용 보조로 둔다.

방법 (합성 시험, 같은 지점)	60분 CO_2 MAE (ppm)	현재값 유지 대비	10~90% 구간 적중률	1회 추론 (배치1, 평균)	GPU 메모리 (가중치 / 평가 중 최대)
현재값 유지	44.0	—	—	—	—
Chronos-Bolt tiny (870만), 과거 일주일	40.6	−8%	0.67	35 ms (GPU)	33 / 240MB
Chronos-Bolt small (4,770만), 과거 일주일	30.4	−31%	0.76	47 ms (GPU)	182 / 705MB
Chronos-Bolt base (2억 530만), 과거 일주일	27.8	−37%	**0.79**	84 ms (GPU)	783 / 961MB
직접 학습 GRU 4시간	**20.5**	**−53%**	—	0.34 ms (ONNX CPU)	수 MB

과거 길이별 60분 CO_2 MAE (ppm, 시험)

과거 길이	tiny	small	base
1시간	53.3	52.9	53.7
4시간	49.1	51.1	48.5
24시간	44.5	46.7	46.1
96시간	43.6	37.7	37.5
일주일 (검증으로 선택)	**40.6**	**30.4**	**27.8**

측정 조건: 합성 시험 2026-10-05 ~ 10-11, 과제 2와 같은 지점(6,046개). 과거 길이는 검증 기간(09-28 ~ 10-04)으로 골랐다. GPU(CUDA 13, FP32), 보드 25W, 클럭 고정 안 함, 워밍업 20회·100회 측정. 평가 묶음은 tiny·small 128개, base 16개(128개에서는 메모리 부족으로 강제 종료).

확인 기준

기준	결과	판정
세 방법을 같은 시점에서 비교	기준선·GRU·Chronos 3종을 같은 평가 지점에서 비교	통과
정확도·구간 적중률·추론 시간·메모리로 판단 근거	위 표의 네 열	통과

산출물

- **비교표와 채택 의견**: 위 표와 해결한 것 5번.

생각해 볼 점

- **직접 학습한 모델과의 차이는 어디서 오나**: Chronos는 CO_2 과거값만 본다. GRU와의 차이(27.8 vs 20.5) 중 일정 몫은 약 3ppm이다(일정 뺀 GRU 23.7). 나머지 약 4ppm은 시각·요일·온습도를 함께 보는지에서 온다.
- **과거를 얼마나 길게 줄 때 가장 잘 맞나**: 일주일이다. 과거 길이가 모델 크기보다 중요하다. small은 24시간 46.7 → 일주일 30.4ppm이다. 요일까지 한 바퀴 봐야 시간표 반복을 스스로 찾는다. 그래서 새 공간의 첫 주처럼 과거가 짧을 때는 현재값 유지와 비슷하다.
- **로컬 LLM과 같이 올릴 수 있나**: base는 프로세스 전체 2.3GB를 쓰고, 이 보드(7.3GB, CPU·GPU 공유)에서 큰 묶음으로 돌리다 강제 종료됐다. LLM과 함께라면 small(GPU 최대 0.7GB)이 한계다.
- **GPU와 CPU**: 정확도는 같다(small 과거 96시간 37.7ppm). 지연만 CPU 2스레드 284ms에서 GPU 47ms로 줄었다.

심화(B)

- **다른 변수(온습도·일정)를 함께 넣기**: 다른 변수를 받을 수 있는 Chronos-2로 실험했다(추가 학습 없음). 온습도와 일정·시각까지 넣자 60분 CO_2 오차가 **17.6ppm**으로, 직접 학습한 GRU(20.5ppm)보다도 좋았다. 온습도만 더하면 거의 그대로였고(21.4 → 21.0ppm), 큰 차이는 일정·시각에서 나왔다. "미리 알 수 있는 일정"이 가장 중요하다는 과제 3의 결론과 같은 방향이다. 다만 한 번 평가에 수십 분이 걸려(시험 기간 약 21분) 서비스에 바로 올리기는 어렵다.

넣은 정보 (Chronos-2, 과거 일주일)	검증 60분	시험 60분	10~90% 구간 적중률 (시험)
CO_2만	27.3ppm	21.4ppm	0.73

넣은 정보 (Chronos-2, 과거 일주일)	검증 60분	시험 60분	10~90% 구간 적중률 (시험)
+ 온습도	27.8ppm	21.0ppm	0.76
+ 온습도 + 일정·시각	**23.7ppm**	**17.6ppm**	0.75
(비교) 직접 학습 GRU 4시간	21.1ppm	20.5ppm	—

측정 조건: 합성 데이터, 검증 09-28 ~ 10-04, 시험 10-05 ~ 10-11, 과제 2와 같은 평가 지점. 과거 길이(24시간·일주일)는 검증으로 골랐다. GPU, 보드 25W.

- **직접 학습 모델과 섞은 예측**: GRU와 Chronos를 반반 섞으면 60분 오차가 오히려 나빠졌다(GRU 20.5 → 21.9~23.0ppm). 검증 기간으로 비중을 고르면 GRU 비중이 0.95로 나오고, 오차는 20.3ppm으로 0.2ppm만 좋아졌다(아래 표). Chronos가 GRU보다 크게 약해서 섞는 이득이 거의 없다. 추론 비용(250배)을 생각하면 섞지 않는 것이 낫다.

섞은 예측 (60분 CO_2 MAE, ppm)

방법	GRU + Chronos small, 검증	시험	GRU + Chronos base, 검증	시험
GRU 단독	21.12	20.52	21.12	20.52
Chronos 단독	37.13	30.40	35.48	27.85
반반 평균	26.22	23.03	25.52	21.85
검증으로 고른 비중 (GRU 0.95)	21.07	**20.35**	21.06	**20.28**

측정 조건: 합성 데이터, 검증 09-28 ~ 10-04, 시험 10-05 ~ 10-11, 같은 평가 지점. Chronos는 과거 일주일. 비중은 검증 기간만으로 골랐다.

과제 7. 엣지 최적화

해결한 것

1. **학습한 모델 5개(예측 GRU·LSTM·TCN, 이상 탐지 오토인코더, 환기 경보 MLP)를 TensorRT FP16으로 변환**했다. GRU·LSTM은 FP16에서 차이가 기준을 조금 넘어, 순환층만 FP32로 두는 혼합 정밀도로 다시 변환했다.
2. **네 가지 실행 방식(PyTorch CPU, PyTorch GPU, ONNX Runtime, TensorRT)의 지연**을 평균과 느린 쪽 5%(p95)로 재다. 배치 1(한 공간)과 배치 4(네 공간)를 모두 재고, 배치 4도 거의 같았다(아래 표는 배치 1).
3. **변환 전후 결과 차이를 원래 단위로** 재다. CO_2 예측은 최대 몇 ppm 달라졌는지, 분류는 판정이 바뀐 비율로 봤다.

4. **측정 중 전력과 메모리**를 기록했다. 보드 전체 전력은 7.2~9.1W였다. TensorRT 최대 배치를 4,096에서 64로 줄이자 실행 메모리가 215MB에서 수 MB로 줄었다.

지연 (ms, 평균 / p95, 배치 1)과 변환 차이

모델	PyTorch CPU	PyTorch GPU	ONNX Runtime CPU	TensorRT FP16	FP16 변환 후 최대 차이
예측 GRU 4시간	32.1 / 36.1	1.18 / 1.30	**0.34 / 0.35**	1.70 / 1.73	CO_2 1.10ppm (평균 0.06)
예측 LSTM 4시간	4.42 / 4.41	1.16 / 1.21	**0.42 / 0.45**	1.66 / 1.71	CO_2 1.03ppm
예측 TCN 4시간	2.68 / 3.11	3.07 / 3.22	**0.23 / 0.25**	0.58 / 0.64	CO_2 0.25ppm
이상 탐지 오토인코더	1.10 / 1.60	1.09 / 1.26	**0.04 / 0.04**	0.40 / 0.42	재구성 0.001
환기 경보 MLP	2.61 / 3.59	0.80 / 0.90	**0.03 / 0.04**	0.32 / 0.34	판정 바뀜 0%

혼합 정밀도로 다시 변환한 결과

모델	정밀도	CO_2 최대 차이 (ppm)	배치 1 평균 / p95 (ms)	엔진 (KB)
GRU	FP16 전체	0.94~1.10 (빌드마다 다름)	1.70 / 1.81	537
GRU	**혼합(순환층 FP32)**	**0.27**	1.74 / 1.82	701
LSTM	FP16 전체	1.01~1.03	1.67 / 1.83	556
LSTM	**혼합(순환층 FP32)**	**0.27**	1.74 / 1.81	749

측정 조건: 보드 25W 모드, 클럭 고정 안 함, 워밍업 50회 뒤 500회 측정. 입력은 합성 시험 기간. TensorRT 수치에는 매 호출의 CPU↔GPU 복사가 들어 있다. 모델마다 새 프로세스로 재다.

확인 기준

기준	결과	판정
변환 후 CO_2 예측 차이 1ppm 이하	TCN 0.25ppm. GRU·LSTM은 FP16 전체에서 1.03~1.10ppm으로 조금 넘었고, 혼합 정밀도로 0.27ppm	통과 (혼합 정밀도 포함)
분류 결과가 바뀐 비율 0.1% 이하	환기 경보 0%	통과
측정 조건(전력 모드, 클럭 고정, 반복, 워밍업) 기록	위 측정 조건	통과

산출물

- **최적화 보고서**: 위 두 표. 결론은 **지금 서비스 규모(공간 3~4개, 1분 주기)라면 ONNX Runtime CPU가 가장 좋다**는 것이다. GPU는 로컬 LLM에 남겨 둔다.

생각해 볼 점

- **CPU로 1ms면 충분한데 왜 변환하나**: 이 크기 모델은 실제로 ONNX Runtime CPU가 가장 빨랐다(0.03~0.42ms). 그래도 변환이 의미 있는 경우는 세 가지다. 로컬 LLM이 CPU를 쓰는 동안 시계열을 GPU로 빼낼 때, 공간이 수십 개로 늘어 한 번에 처리할 양이 커질 때, 같은 일을 더 적은 전력으로 할 때다.
- **순환망(GRU·LSTM)과 합성곱(TCN) 중 무엇이 더 빨라지나**: TCN만 빨라졌다(PyTorch GPU 3.07 → TensorRT 0.58ms). GRU·LSTM은 오히려 느려졌다(1.2 → 1.7ms). 합성곱은 한 번에 병렬로 계산되지만, 순환망은 48칸을 하나씩 차례로 계산해야 해서 GPU의 이득이 작다. 같은 이유로 순환망은 반정밀도 반올림 오차가 48칸을 지나며 쌓여 차이가 커졌다.
- **평균보다 느린 쪽 5%가 중요한 이유**: 상시 서비스는 매분 정해진 시각에 결과를 내야 한다. 평균이 빨라도 가끔 수백 ms씩 튀면 그 주기의 결과가 늦거나 다른 작업과 겹친다. PyTorch CPU GRU는 배치 4에서 평균 36ms에 p95 68ms로 흔들림이 가장 컸다.

심화(B)

- **INT8 변환**: 예측 TCN·GRU·LSTM과 이상 탐지 오토인코더를 INT8로 변환했다(학습 기간 입력 1,024개로 보정). 정확도 손실은 사실상 없었다. 60분 CO_2 오차 차이가 0.002ppm 이하이고, 이상 판정이 바뀐 비율은 0%다. 대신 속도 이득도 없었다. 모델이 아주 작아(1회 0.3~1.8ms) 계산보다 실행 준비와 메모리 복사가 대부분이고, GRU·LSTM의 순환층은 INT8로 바뀌지 않고 FP16으로 남았기 때문이다. 이 크기의 모델은 INT8을 쓸 이유가 없고(빌드만 TCN 213초), INT8은 Chronos처럼 큰 모델에 시도할 가치가 있다.
- **모델 4개를 동시에 올렸을 때의 메모리와 로컬 LLM 조합**: 예측 GRU·오토인코더·사용 여부·환기 경보를 한 프로세스에 올려 세 방식으로 재 보았다(아래 표). 모델 자체는 100MB 안팎이고, 메모리의 대부분은 PyTorch·TensorRT 같은 라이브러리가 쓴다. **네 모델은 ONNX Runtime CPU로 돌리고, GPU와 남는 메모리는 LLM에 맡기는 조합**을 제안한다. 이때 가용 메모리는 약 2.9~3.2GB로, 4비트 양자화 1~2B 모델(약 1.5GB)은 들어가고 1GB 이상 남는다. 3B 4비트(약 2GB 이상)는 여유가 0.5GB 아래로 떨어져 위험하다. 이 보드는 스왑이 없어 메모리가 모자라면 멈추므로, 가용 1GB 이상을 항상 남기는 것을 조건으로 둔다.

INT8 변환 결과 (60분 CO_2 오차와 1회 추론 시간, 배치 1 평균)

모델	FP32	FP16	INT8	FP32 대비 최대 차이 (INT8)
TCN	20.98ppm · 0.47ms	20.98ppm · 0.46ms	20.98ppm · 0.47ms	CO_2 0.39ppm

모델	FP32	FP16	INT8	FP32 대비 최대 차이 (INT8)
GRU	21.28ppm · 1.68ms	21.28ppm · 1.78ms	21.28ppm · 1.81ms	CO_2 1.45ppm
LSTM	22.85ppm · 1.67ms	22.84ppm · 1.71ms	22.84ppm · 1.72ms	CO_2 1.01ppm
오토인코더	0.31ms	0.30ms	0.33ms	이상 판정 바뀜 0%

측정 조건: 합성 데이터 시험 기간 전체 샘플(예측 6,201개, 오토인코더 창 8,064개). 여기 오차는 전체 샘플 기준이라 과제 3 표(평가 지점 기준)와 조금 다르다. 보드 25W, 클럭 고정 안 함, 워밍업 50회·500회 측정, 엔진마다 새 프로세스.

모델 4개 동시 메모리

방식	라이브러리만	모델 4개 올린 뒤	보드 가용 메모리 감소	4개 한 바퀴(배치 4)
PyTorch CPU (지금 서비스)	548MB	658MB	330MB	54.2ms
ONNX Runtime CPU	166MB	**176MB**	**111MB**	60.8ms
TensorRT FP16	582MB	762MB	286MB	17.2ms

측정 조건: 프로세스 메모리(RSS)와 보드 가용 메모리(`free`). 보드 전체 7,485MB, 측정 때 다른 서비스가 떠 있는 상태에서 가용 약 2.7GB(2026-10-21 17:38). 사용 여부 모델은 결정트리 계열이라 세 방식 모두 CPU.

과제 8. 상시 추론 서비스

해결한 것

1. **매분 자동으로 공간(실습실·회의실·사무실)마다 세 가지를 계산**한다.
   - 60분 예측(CO_2·온도)과 "약 N분 뒤 1,000ppm 초과 예상" 경보
   - 이상 여부와 점수, 원인 센서, 규칙 경보(데이터 끊김·값 고정·범위 밖)
   - 사용 여부와 환기 선제 경보
2. **두 가지 방식으로 제공**한다.
   - MQTT: 토픽 `team2-2/inference/<lab|meeting|office>` (포트 1884). 마지막 값을 브로커에 남겨 두는 방식(retain)이라 새로 접속한 구독자도 최신값을 바로 받는다.

- 조회 API: `http://172.27.91.139:8090`. 예를 들어 `/spaces/meeting/co2?minutes=30`으로 "회의실 30분 뒤 CO_2"를 묻는다.

3. **학습 때와 똑같은 전처리**를 거친다. 서비스는 학습과 같은 코드(`prep.py`, `dataset.py`)를 그대로 쓴다. 전처리가 다르면 모델은 학습 때 본 적 없는 값을 받는다. 예를 들어 칸 이름을 시작 시각으로 바꾸면 5분 어긋나고, 정규화를 다시 계산하면 값의 크기가 달라진다. 그러면 오류 없이 조용히 틀린 예측을 낸다.
4. **장애에 버틴다.**
   - 데이터가 일부 끊겨도 서비스는 계속 돌고 `data_gap` 경보로 알린다. 끊김 기준은 공간별 센서 주기에 맞춰 실습실·사무실 5분, 회의실 90분이다.
   - 한 번 계산이 실패해도 오류를 기록하고 다음 주기에 다시 계산한다. 프로세스가 죽으면 감시 스크립트가 10초 뒤 다시 띄운다.
   - 보드를 재부팅하면 자동으로 시작한다(cron `@reboot`).
5. **결과 형식을 문서로 남겼다**: `결과 형식 문서 (다른 팀용)` 탭. 토픽·필드·단위·예시가 있다.

장애 시험 (수집기 10분 정지, 2026-10-20 17:32 ~ 17:42)

시각	일어난 일
17:39	실습실·사무실에 `data_gap` 경보, 알림 1회
17:42	수집기 재시작
17:43~44	경보가 풀리고 정상으로 돌아옴

그동안 서비스는 멈추지 않았다. 시험에서 문제 두 가지를 찾아 고쳤다. 끊김 기준이 10분이라 경보가 늦어서 공간별 기준으로 바꿨고, 값이 빠진 동안 오토인코더가 이상 경보를 같이 내서, 값이 빠져 있으면 이상 점수를 내지 않게 했다.

확인 기준

기준	결과	판정
12시간 무중단 가동 기록	2026-10-20 17:30부터 약 15시간 919주기 연속, 계산 실패 0회	통과
수집기 10분 정지 시 데이터 끊김 경보, 다시 켜면 정상 복귀	위 장애 시험	통과
조회 API로 공간별 예측·이상·상태를 받을 수 있다	다른 노트북(교육장 Wi-Fi)에서 문서만 보고 브라우저·curl로 받음 (2026-10-21 14:13)	통과
결과 형식 문서(토픽·필드·단위·예시)	결과 형식 문서 탭	통과

산출물

- **예측 서비스 v1**: 팀 Jetson에서 가동 중. 실공간용 예측 모델은 공간·일정 입력을 뺀 범용 모델이다(합성 시험 60분 CO_2 28.6ppm). 실데이터 학습 모델로 바꾸는 것이 다음 단계다.
- **12시간 가동 기록 요약**
  - 주기 처리 시간: 평균 0.72초, p95 0.85초, 최대 2.6초(과제 7 측정과 겹친 때)
  - 경보: 켜져 있던 시간은 사무실 이상 476분, CO_2 높음 41분, 실습실 값 고정 16분 등 모두 573분이다. 실제로 보낸 알림은 33건이다.
  - 장애 시험: 위 표
- **결과 형식 문서**: `결과 형식 문서 (다른 팀용)` 탭.

측정 조건: 실데이터 실시간, 2026-10-20 17:30 ~ 10-21 08:47, 보드 25W.

생각해 볼 점

- **막 시작된 5분 칸을 써도 되나**: 쓰지 않는다. 1~2분치만 든 칸은 평균이 그 몇 건에 좌우되고, 학습 때는 그런 칸이 없었다. 끝난 칸까지만 쓰고, 결과의 `time`은 그 칸의 끝 시각이다.
- **같은 고장에 매분 경보를 보내면**: 운영자가 알림을 끄게 된다. 같은 경보는 30분에 한 번만 다시 보내고, 풀릴 때 한 번 알린다. 매분 보냈다면 573건, 실제로는 33건이었다.
- **로컬 LLM이 이 서비스를 도구로 쓰려면**: 지금은 "N분 뒤 CO_2"와 경보를 준다. 더 필요한 것은 세 가지다. 예측을 얼마나 믿을지(예측 구간, 최근 예측 오차), 왜 이상인지의 근거(원인 센서의 최근 값), 단위·시각이 붙은 짧은 문장 요약이다.

심화(B)

- **예측으로 환기를 미리 켜는 규칙과 지난 1주 적용**: “30분 안 예측 최댓값이 1,000ppm 이상이면 환기를 켜고, 900ppm 밑으로 내려가면 끈다(최소 15분 유지)” 규칙을 만들었다. 합성 시험 주간에 적용해 보니 1,000ppm을 넘은 시간이 2,466분에서 376분으로 85% 줄었다. 지금 값을 보고 켜는 규칙(504분)보다도 25% 적다. 대신 환기 시간이 약 26% 늘어 냉난방 에너지가 더 든다(아래 표).
- **서비스가 스스로 성능을 감시**: 서비스가 매 주기 낸 60분 예측을 저장해 두었다가, 실제 값이 들어오면 채점한다. 최근 6시간의 60분 앞 CO_2 오차가 기준(검증 오차의 2배, 68.6ppm)을 넘으면 `forecast_degraded` 경보를 낸다. 기존 API·MQTT 필드는 그대로 두고, 공간 결과에 `accuracy` 필드와 새 주소 `/spaces/{공간}/accuracy`를 더했다. 만든 직후 실습실(105.8ppm)과 사무실(165.4ppm)에서 경보가 켜졌다. 서비스 모델이 합성 데이터로 학습한 범용 모델이라 실공간에서 검증 오차의 3~5배가 나오기 때문이고, 감시 기능이 의도대로 작동한 것이다. 할 일은 실데이터로 학습한 모델로 바꾸고 기준도 실데이터 검증 오차로 다시 정하는 것이다.

환기 제어 시뮬레이션: 1,000ppm 이상인 시간 (환기 강화 +2회/시간)

공간	제어 없음 (분)	지금 값으로 켬 (분)	예측으로 미리 켬 (분)	줄어든 비율	켜진 횟수
강의실 A	845	191	**164**	81%	24
강의실 B	1,432	234	**161**	89%	43
회의실	52	10	**10**	81%	2
라운지	137	69	**41**	70%	4
합계	**2,466**	504	**376**	**85%**	73

측정 조건: 합성 시험 주간(10-05 ~ 10-11), 1분 단위 실제 CO_2. 환기 효과는 “바깥 공기(420ppm)와 섞이는 비율이 늘어난다”는 간단한 공기 교환 식으로 계산했다. 이 식을 데이터에 맞추자 환기율이 생성기 설정과 거의 같게 나왔다. 예측은 과제 3 모델의 저장된 결과를 썼다. 사람 수와 고장은 그대로 두고 환기만 바꾼다는 가정이다.

환기를 더 세게(+4회/시간) 하면 예측형은 93%까지 줄고, 약하게(+1회) 하면 68%다. 강의실 B는 수업 중 계속 높아 +2회로도 161분이 남으므로, 수업 시작 전 미리 환기하는 운영 조치가 함께 필요하다.

자기 성능 감시 응답 예 (`GET /spaces/lab/accuracy`, 2026-10-21 15:50, 일부 생략)

```
{"space": "실습실", "window_hours": 6.0, "as_of": "2026-10-21T15:50:00",
 "mae_co2_60_ppm": 105.8, "n_60": 71, "threshold_co2_60_ppm": 68.6,
 "degraded": true, "enough_data": true,
 "by_horizon": [{"minutes_ahead": 60, "n": 71, "mae_co2_ppm": 105.8,
"bias_co2_ppm": 3.2}]}
```

측정 조건: 실데이터, 2026-10-21 약 09:45 ~ 15:50의 60분 앞 예측. 채점은 그 5분 칸에 실제 메시지가 있는 값만 쓴다(메운 값 제외).

실데이터 재평가 (과제 2·3 추가)

시간표 없이 학습한 GRU(과거 2시간, 검증 기준 선택)의 60분 CO_2 오차는 38.1ppm으로, 현재값 유지(46.4)보다 18% 작다. 기준(30%)에는 미달이다. 온도는 TCN 계열(0.09℃)이 현재값 유지(0.13)를 넘는다.

방법 (실데이터 시험 MAE)	CO_2 5분	15분	30분	60분	온도 60분	검증 CO_2 60분
현재값 유지	7.6	14.5	27.2	46.4	0.13	—
GRU 2시간 (검증 기준 선택)	**7.0**	**11.9**	**22.9**	**38.1**	0.18	**20.4**
GRU 4시간	6.9	12.2	23.5	40.7	0.093	24.0
TCN 2시간	7.0	12.2	23.2	40.1	0.089	20.6
TCN 4시간	7.0	12.0	23.1	40.0	0.090	22.3

측정 조건: 실데이터, 학습 2025-08-01 ~ 2026-09-21(약 20만 샘플), 검증 09-22 ~ 10-05, 시험 10-06 ~ 10-20, GPU 학습. 공간은 실습실·사무실·회의실(약 30분마다 보냄).

- **왜 합성 데이터만큼 안 되나**: 실데이터에는 시간표가 없다. 합성에서 일정을 빼면 오차가 14% 늘고, 사용 중 회의실은 두 배가 됐다. 5~7월(약 160만 건)을 더 넣어도 결과는 학습 흔들림 범위에서만 움직였다. **데이터 양보다 빠진 정보(일정)가 병목**이다.
- **해야 할 일**: 교육장 시간표와 회의실 예약을 받아 입력에 넣는다.
- **모델 사이 1~3ppm 차이는 학습 흔들림 안**이라 순위를 말하기 어렵다. 같은 TCN을 두 번 학습하면 최대 1.2ppm 달랐다.

가장 의외였던 발견

게이트웨이 한 대의 시계가 4분 55초 빨랐다. 그 게이트웨이를 거친 센서(사무실·회의실 전부와 실습실 일부)의 데이터가 미래 시각으로 들어오고 있었다. 수집기가 받은 시각과의 차이(`rx_delay_s`)를 함께 남겨 두지 않았다면 발견하지 못했을 것이다. 그대로 학습했다면 사무실 CO_2가 실습실보다 5분 먼저 반응하는 것처럼 보였을 것이다. 설정 한 줄(네트워크 서버 시각 `nsTime`)로 고쳤다. **데이터의 "시각"도 측정값처럼 검증해야 한다.**

평가 기준 자체 점검

영역 (비중)	근거	판정
동작 (25%)	네트워크 차단·재부팅·수집기 10분 정지 시험 통과, 919주기 실패 0	우수
성능 (25%)	과제 1~8 확인 기준 모두 충족. 실데이터 60분 예측은 18%로 미달(원인·해야 할 일 제시)	우수

영역 (비중)	근거	판정
해석 (30%)	과제마다 “생각해 볼 점”에 답하고, 못 하는 것(냉방 정지, 관측 불가 고장, 값 고정, 라운지, 일정 없는 실데이터)을 짚음	우수
공정한 평가 (10%)	같은 평가 지점, 시험 기간 미사용, 학습 기간 정규화, 모든 표에 측정 조건, 생성기 버그 수정 공개	우수
재사용성 (10%)	결과 형식 문서, MQTT retain + 조회 API, 다른 노트북에서 문서만 보고 접속 성공	우수

심화 항목: 안내문의 심화 항목을 모두 시도했다(과제 1, 3, 4, 5, 6, 7, 8). 과제 5의 노쇼·예약 없는 사용 찾기는 표본이 3~6건뿐이라 결론을 조심해서 봐야 한다.